제약 시스템

개요

제약 시스템은 R1CS 제약 조건들의 집합을 관리하고 검증하는 구조입니다. 산술 회로의 정확성을 보장하는 핵심 메커니즘을 제공합니다.

R1CS 제약 시스템 클래스

기본 구조

libiop에서 R1CS 제약 시스템은 `r1cs_constraint_system` 클래스로 구현됩니다:

```
template<typename FieldT>
class r1cs_constraint_system {
public:
size_t primary_input_size_; // 공개 입력 변수 수
size_t auxiliary_input_size_; // 보조 변수 수
std::vector<r1cs_constraint<FieldT>> constraints_; // 제약 조건들
// 디버그 정보 (DEBUG 모드에서만)
#ifdef  DEBUG
std::map<size_t, std::string> constraint_annotations_; // 제약 조건 주석
std::map<size_t, std::string> variable_annotations_; // 변수 주석
#endif
```

```
// 생성자

r1cs_constraint_system() : primary_input_size_(0), auxiliary_input_size_(0) {}

// 시스템 정보 조회

size_t  num_inputs() const; // 공개 입력 수

size_t  num_variables() const; // 전체 변수 수

size_t  num_constraints() const; // 제약 조건 수

// 유효성 검증

bool  is_valid() const;

// 제약 조건 만족 여부 확인

bool  is_satisfied(const r1cs_primary_input<FieldT> &primary_input,

const r1cs_auxiliary_input<FieldT> &auxiliary_input) const;

bool  is_satisfied(const r1cs_variable_assignment<FieldT> &full_variable_assignm

// 제약 조건 추가

void  add_constraint(const r1cs_constraint<FieldT> &c);

void  add_constraint(const r1cs_constraint<FieldT> &c, const std::string &annota

// 비교 연산자

bool  operator==(const r1cs_constraint_system<FieldT> &other) const;

// 메모리 사용량 계산

std::size_t  size_in_bytes() const;

};
```

시스템 정보 메서드 구현

```
template<typename FieldT>
size_t r1cs_constraint_system<FieldT>::num_inputs() const {
return primary_input_size_;
}

template<typename FieldT>
size_t r1cs_constraint_system<FieldT>::num_variables() const {
return primary_input_size_ + auxiliary_input_size_;
}

template<typename FieldT>
size_t r1cs_constraint_system<FieldT>::num_constraints() const {
return  constraints_.size();
}
```

제약 조건 생성 인터페이스

```
template<typename FieldT>
class ConstraintBuilder {
private:
R1CSConstraintSystem<FieldT>& cs;
public:
ConstraintBuilder(R1CSConstraintSystem<FieldT>&  constraint_system)
```

```
: cs(constraint_system) {}

// 등식 제약: lhs = rhs

void  enforce_equal(

const LinearCombination<FieldT>&  lhs,

const LinearCombination<FieldT>&  rhs

) {

LinearCombination<FieldT> a = lhs - rhs;

LinearCombination<FieldT> b;

b.add_constant(FieldT::one());

LinearCombination<FieldT> c; // 0

cs.add_constraint(R1CSConstraint<FieldT>(a, b, c));

}

// 곱셈 제약: result = left * right

void  enforce_multiplication(

size_t  left_var,

size_t  right_var,

size_t  result_var

) {

LinearCombination<FieldT> a, b, c;

a.add_term(left_var, FieldT::one());

b.add_term(right_var, FieldT::one());

c.add_term(result_var, FieldT::one());

cs.add_constraint(R1CSConstraint<FieldT>(a, b, c));
```

```
}

// 불린 제약: var * (1 - var) = 0

void  enforce_boolean(size_t  var) {

LinearCombination<FieldT> a, b, c;

a.add_term(var, FieldT::one());

b.add_constant(FieldT::one());

b.add_term(var, -FieldT::one());

// c는 0 (기본값)

cs.add_constraint(R1CSConstraint<FieldT>(a, b, c));

}

// 조건부 선택: result = condition ? if_true : if_false

size_t  conditional_select(

size_t  condition_var,

size_t  if_true_var,

size_t  if_false_var

) {

size_t result_var = cs.allocate_aux();

size_t temp_var = cs.allocate_aux();

// temp = condition * (if_true - if_false)

LinearCombination<FieldT> a, b, c;

a.add_term(condition_var, FieldT::one());

b.add_term(if_true_var, FieldT::one());

b.add_term(if_false_var, -FieldT::one());
```

```
c.add_term(temp_var, FieldT::one());

cs.add_constraint(R1CSConstraint<FieldT>(a, b, c));

// result = temp + if_false

LinearCombination<FieldT> a2, b2, c2;

a2.add_term(temp_var, FieldT::one());

a2.add_term(if_false_var, FieldT::one());

b2.add_constant(FieldT::one());

c2.add_term(result_var, FieldT::one());

cs.add_constraint(R1CSConstraint<FieldT>(a2, b2, c2));

return result_var;

}

};
```

제약 조건 검증 알고리즘

시스템 유효성 검증

제약 시스템 자체의 유효성을 먼저 확인합니다:

```
template<typename FieldT>

bool r1cs_constraint_system<FieldT>::is_valid() const {

// 입력 변수 수가 전체 변수 수를 초과하지 않는지 확인

if (this->num_inputs() > this->num_variables()) return false;
```

```
// 각 제약 조건의 선형 결합이 유효한지 확인
for (size_t c = 0; c < constraints_.size(); ++c) {
if (!(constraints_[c].a_.is_valid(this->num_variables()) &&
constraints_[c].b_.is_valid(this->num_variables()) &&
constraints_[c].c_.is_valid(this->num_variables()))) {
return false;
}
}
return true;
}
```

제약 조건 만족 여부 검증

공개/보조 입력으로부터 검증

```
template<typename FieldT>
bool r1cs_constraint_system<FieldT>::is_satisfied(
const r1cs_primary_input<FieldT> &primary_input,
const r1cs_auxiliary_input<FieldT> &auxiliary_input
) const {
assert(primary_input.size() == num_inputs());
assert(primary_input.size() + auxiliary_input.size() == num_variables());

// 전체 변수 할당 생성
```

```
const r1cs_variable_assignment<FieldT> full_variable_assignment =
variable_assignment_from_inputs(primary_input, auxiliary_input);
return  this->is_satisfied(full_variable_assignment);
}
```

전체 변수 할당으로부터 검증

```
template<typename FieldT>
bool r1cs_constraint_system<FieldT>::is_satisfied(
const r1cs_variable_assignment<FieldT> &full_variable_assignment
) const {
for (size_t c = 0; c < constraints_.size(); ++c) {
// 각 선형 결합 평가
const FieldT ares = constraints_[c].a_.evaluate(full_variable_assignment);
const FieldT bres = constraints_[c].b_.evaluate(full_variable_assignment);
const FieldT cres = constraints_[c].c_.evaluate(full_variable_assignment);

// R1CS 조건 확인: A * B = C
if (!(ares * bres == cres)) {
#ifdef  DEBUG
// 디버그 정보 출력
auto it = constraint_annotations_.find(c);
printf("constraint %zu (%s) unsatisfied\n", c,
```

```
(it == constraint_annotations_.end() ? "no annotation" : it->second.c_str()));

printf("<a,(1,x)> = ");  ares.print();

printf("<b,(1,x)> = ");  bres.print();

printf("<c,(1,x)> = ");  cres.print();

#endif

return false;

}

}

return true;

}
```

변수 할당 생성 유틸리티

```
template<typename FieldT>

r1cs_variable_assignment<FieldT> variable_assignment_from_inputs(

const r1cs_primary_input<FieldT> &primary_input,

const r1cs_auxiliary_input<FieldT> &auxiliary_input

) {

// 공개 입력과 보조 입력을 연결하여 전체 할당 생성

r1cs_variable_assignment<FieldT> full_variable_assignment = primary_input;

full_variable_assignment.insert(full_variable_assignment.end(),

auxiliary_input.begin(),

auxiliary_input.end());
```

```
return full_variable_assignment;

}
```

병렬 검증

```
template<typename FieldT>

class ParallelR1CSVerifier {

public:

static bool verify_system_parallel(

const R1CSConstraintSystem<FieldT>& cs,

const vector<FieldT>& assignment

) {

const size_t num_constraints = cs.num_constraints();

atomic<bool> all_satisfied(true);

#pragma omp parallel for

for (size_t i = 0; i < num_constraints; i++) {

if (all_satisfied.load()) { // 조기 종료 최적화

if (!verify_constraint(cs.get_constraint(i), assignment)) {

all_satisfied.store(false);

}

}

}

return all_satisfied.load();
```

```
}

};
```

희소 행렬 표현과 최적화

libiop의 희소 행렬 구현

libiop에서는 R1CS 시스템을 희소 행렬로 변환하는 기능을 제공합니다:

```
template<typename FieldT>

using naive_sparse_matrix = std::vector<std::map<std::size_t, FieldT>>;
```

A, B, C 행렬 생성

각 제약 조건의 선형 결합을 행렬 형태로 변환합니다:

```
template<typename FieldT>

naive_sparse_matrix<FieldT> r1cs_constraint_system<FieldT>::A_matrix() const {

naive_sparse_matrix<FieldT> matrix;

for (std::size_t i = 0; i < this->num_constraints(); ++i) {

const r1cs_constraint<FieldT> &constraint = this->constraints_[i];



std::vector<linear_term<FieldT>> terms = constraint.a_.terms;

std::map<std::size_t, FieldT> values;

// 각 선형 항을 행렬 원소로 변환

for (std::size_t j = 0; j < terms.size(); ++j) {
```

```
values.insert(std::pair<std::size_t, FieldT>(terms[j].index_, terms[j].coeff_));
}

matrix.push_back(std::move(values));
}
return matrix;
}
```

B_matrix()와 C_matrix()도 동일한 방식으로 구현됩니다.

행렬-벡터 곱셈 최적화

효율적인 Az, Bz, Cz 계산을 위한 전용 메서드:

```
template<typename FieldT>
void r1cs_constraint_system<FieldT>::create_Az_Bz_Cz_from_variable_assignment(
const r1cs_variable_assignment<FieldT> &variable_assignment,
std::vector<FieldT> &Az_out,
std::vector<FieldT> &Bz_out,
std::vector<FieldT> &Cz_out
) const {
// 변수 할당 z는 (1, v, w) 구조로 가정
for (size_t i = 0; i < this->constraints_.size(); ++i) {
// A_i * z 계산
FieldT Az_i = FieldT::zero();
```

```
for (auto &lt : this->constraints_[i].a_.terms) {
Az_i += variable_assignment[lt.index_] * lt.coeff_;
}
Az_out.emplace_back(Az_i);

// B_i * z 계산
FieldT Bz_i = FieldT::zero();
for (auto &lt : this->constraints_[i].b_.terms) {
Bz_i += variable_assignment[lt.index_] * lt.coeff_;
}
Bz_out.emplace_back(Bz_i);

// C_i * z 계산
FieldT Cz_i = FieldT::zero();
for (auto &lt : this->constraints_[i].c_.terms) {
Cz_i += variable_assignment[lt.index_] * lt.coeff_;
}
Cz_out.emplace_back(Cz_i);
}
}
```

메모리 사용량 분석

시스템의 메모리 효율성을 평가하는 메서드:

```
template<typename FieldT>
std::size_t r1cs_constraint_system<FieldT>::size_in_bytes() const {
std::size_t num_terms = 0;

// 모든 제약 조건의 항 수 계산
for (auto &C : this->constraints_) {
num_terms += (C.a_.terms.size() +
C.b_.terms.size() +
C.c_.terms.size());
}

/*
가장 효율적인 표현 방식 가정:
- 제약 조건당 3비트 (A, B, C 구분용)
- 항당 log2(num_variables) + |FieldT| 비트
*/
const std::size_t size_in_bits = 3 * this->num_constraints() +
num_terms * (libff::log2(this->num_variables()) + 8*sizeof(FieldT));

return size_in_bits / 8;
}
```

희소성 활용 최적화

libiop의 희소 행렬 표현은 다음과 같은 장점을 제공합니다:

1. **메모리 효율성**: 0이 아닌 원소만 저장
2. **캐시 친화적**: 연속된 메모리 접근 패턴
3. **병렬화 가능**: 각 제약 조건을 독립적으로 처리

메모리 최적화

지연 평가 시스템

```
template<typename FieldT>
class LazyR1CSSystem {
private:
vector<function<R1CSConstraint<FieldT>()>> constraint_generators;
public:
// 제약 조건 생성기 추가
void add_constraint_generator(
function<R1CSConstraint<FieldT>()> generator
) {
constraint_generators.push_back(generator);
}
// 스트리밍 검증
bool verify_streaming(const vector<FieldT>& assignment) const {
for (const auto& generator : constraint_generators) {
```

```
R1CSConstraint<FieldT> constraint = generator();
if (!verify_constraint(constraint, assignment)) {
return false;
}
}
return true;
}
};
```

배치 처리 최적화

```
template<typename FieldT>
class BatchR1CSProcessor {
public:
// 여러 할당을 동시에 검증
vector<bool> verify_batch(
const R1CSConstraintSystem<FieldT>&  cs,
const vector<vector<FieldT>>&  assignments
) const {
vector<bool> results(assignments.size());
#pragma  omp  parallel  for
for (size_t i = 0; i < assignments.size(); i++) {
results[i] = R1CSVerifier<FieldT>::verify_system(
```

```
cs, assignments[i]
).is_satisfied;
}
return results;
}
};
```

디버깅과 분석 도구

제약 조건 분석기

```
template<typename FieldT>
class R1CSAnalyzer {
public:
struct SystemStatistics {
size_t num_constraints;
size_t num_variables;
size_t num_inputs;
size_t num_aux;
double sparsity_ratio;
size_t max_degree;
};
static SystemStatistics analyze_system(
```

```
const R1CSConstraintSystem<FieldT>&  cs
) {
SystemStatistics stats;
stats.num_constraints = cs.num_constraints();
stats.num_variables = cs.num_vars();
stats.num_inputs = cs.num_inputs();
stats.num_aux = cs.num_aux_vars();
// 희소성 비율 계산
size_t total_entries = stats.num_constraints * stats.num_variables * 3;
size_t non_zero_entries = count_non_zero_entries(cs);
stats.sparsity_ratio = 1.0 - (double)non_zero_entries / total_entries;
return stats;
}
// 제약 조건 의존성 그래프 생성
static vector<vector<size_t>> build_dependency_graph(
const R1CSConstraintSystem<FieldT>&  cs
) {
vector<vector<size_t>> graph(cs.num_vars());
for (size_t i = 0; i < cs.num_constraints(); i++) {
const  auto& constraint = cs.get_constraint(i);
auto a_vars = extract_variables(constraint.get_a());
auto b_vars = extract_variables(constraint.get_b());
auto c_vars = extract_variables(constraint.get_c());
```

```
// 변수 간 의존성 추가
for (size_t a_var : a_vars) {
for (size_t c_var : c_vars) {
graph[a_var].push_back(c_var);
}
}
for (size_t b_var : b_vars) {
for (size_t c_var : c_vars) {
graph[b_var].push_back(c_var);
}
}
}
return graph;
}
};
```

다음 단계

다음 문서에서는 변수 할당을 통해 증인 생성과 변수 인덱싱 방법을 자세히 살펴보겠습니다.

← 이전: R1CS 정의 | 다음: 변수 할당 →